Supplementary Material for EviTrace

Evidence-Guided Requirements-to-Code Link Recovery
for Collaborative Software Maintenance

Code and supplementary material: https://github.com/Wzhgeek/EviTrace

This supplement specifies the evaluated artifacts, final configuration, prompts, gate calibration, comparison methods, complete component results, and statistical procedures. Symbols and method names follow the main manuscript.

S1 Projects and evaluated artifacts

The original collection contains 5,950 candidate requirements-to-code pairs. Four complete requirement groups used during method development account for 168 pairs and are excluded. The evaluation contains 5,782 pairs from 448 requirements across eight projects, with 2,891 positive and 2,891 negative pairs. The development groups comprise two requirements from Dronology and two from Maven. Class balance is imposed within each project. Requirements without positive candidates are retained in classification.

Table S1: Evaluated project scope. Requirements and code are artifact counts.

Project	Req. language	Code language	Req.	Code	Positive	Negative
eTour	English	Java	58	116	308	308
eAnci	Italian	Java	137	55	567	567
iTrust	English	Java/JSP	34	137	255	255
SMOS	English	Java/HTML	67	99	1,044	1,044
LibEST	English	C	47	11	204	204
Albergate	Italian	Java	16	52	53	53
Dronology	English	Java	55	184	314	314
Maven	English	Java	34	82	146	146
Total			448	736	2,891	2,891

Public requirements, code artifacts, and reference links are obtained from the corresponding traceability replication materials. Six projects retain their existing candidate identities. Dronology and Maven retain all distributed positive links and deterministically sampled non-gold negatives within requirements. The evaluated code artifacts are files, classes, or JSP/HTML artifacts. Text is decoded as UTF-8 with ISO-8859-1 fallback, line endings are normalized, and outer whitespace is removed.

A shared external library contains 7,980 labeled pairs: 4,284 from Derby, 1,680 from Drools, 1,092 from Pig, and 924 from Seam2. Positive and negative pairs number 3,990 each. All eight evaluation projects are excluded from the library and gate calibration. Source–target overlap is checked by artifact hashes and normalized text. The eight evaluation projects were established after exploratory pilots. Statistical intervals describe this fixed project collection.

S2 Final model and code context

Table S2: Shared service and output settings.

Setting	Final configuration
Chat model alias	`deepseek-flash`
Recorded service version	DeepSeek-V4.1-Flash-1017
Temperature / native reasoning / streaming	0 / disabled / disabled
Summary output limit	1,024 tokens
Structured-evidence output limit	768 tokens
Direct yes/no output limit	16 tokens
LiSSA reasoning output limit	1,536 tokens
Embedding model / dimensions	text-embedding-v4 / 1,024
Valid responses per complete request	One

The same chat model, temperature, and native-reasoning mode are used for summaries, same-model baselines, and matched controls. The service response exposes the model alias. Identical complete request payloads reuse the same valid cached response. A response is accepted only when its finish reason is `stop`, the response model matches the request, and the output satisfies the corresponding parser. Errors and incomplete responses remain invalid predictions.

A requirement-independent English summary $s(c)$ is generated from complete normalized code and reused across candidate pairs. The summary prompt requests an overall description and coarse-grained functional parts, within 350 words and ten bullet points. The word and bullet limits are provided as prompt guidance. Original source evidence $e(r, c)$ is selected for the current requirement r and retains original line numbers.

Let L be the source-line count, T_ℓ the code terms on line ℓ, and Q_r the requirement terms after identifier splitting and stop-word and generic-term removal. Each line is scored by

$$a_\ell = \frac{d_\ell}{\sqrt{|T_\ell| + 1}} \sum_{t \in Q_r \cap T_\ell} \left(1 + \log \frac{L + 1}{df_{\text{line}}(t) + 1}\right). \tag{S1}$$

Here $df_{\text{line}}(t)$ counts source lines containing t. The declaration factor d_ℓ is 1.3 for a non-comment declaration and 1 otherwise. Terms are summed in deterministic order and equal scores are resolved by source order.

The evidence budget is 8,000 rendered characters. Source is retained in full for 4,413 evaluated pairs (76.32%). Longer artifacts are represented using declaration anchors, the 24 highest-scoring positions with three lines on either side, and fixed declaration windows. Up to four type headers and twelve relevance-ranked declaration windows are considered, with additional declaration positions distributed across the artifact. Complete lines are retained within the budget, deduplicated, and restored to source order. The final context is $z(r, c) = s(c) \parallel e(r, c)$. The summary provides a functional overview, while the original passages supply the code details used for the decision.

The source budget and local windows control input size and preserve nearby context. The 1.3 factor gives declaration lines a 30% base-score boost. These settings are applied uniformly across projects.

S3 Dense retrieval and source-text gate calibration

A dense query concatenates the complete original requirement and original code. The same query representation is used across the matched retrieval comparisons. Exact cosine retrieval returns the global top three cases, retaining only scores strictly above 0.5. These two settings bound the number and similarity of the supplied examples.

Queries estimated to exceed 30,000 tokens or 120,000 UTF-8 bytes are split into complete UTF-8 chunks of at most 24,000 bytes. The Qwen3-Embedding-0.6B tokenizer is used for the token estimate. Each chunk vector is normalized, the vectors are averaged with weights equal to chunk byte lengths, and the result is normalized. Every query byte is covered. The archived case vectors include 6,433 full-document matches, 1,533 source-prefix matches, and 14 encoding/whitespace-equivalent or prefix matches. These fixed vectors and the corresponding hits are shared in the matched retrieval comparisons.

The bilateral gate compares requirements with requirements and original code with original code. Identifier boundaries are split, Unicode text is normalized, and case is folded. Function words are removed on both sides, with programming-language keywords additionally removed on the code side. Binary term vectors are weighted by source-only inverse document frequency:

$$w_x(t) = 1 + \log \frac{N_x + 1}{df_x(t) + 1}, \qquad x \in \{r, c\}. \tag{S2}$$

Each distinct nonempty external artifact is counted once. The calibration corpus contains 849 requirements artifacts and 2,883 code artifacts. Unseen terms receive weight $1 + \log(N_x + 1)$ and contribute to the vector norm.

For each reference pair, two different source projects are drawn uniformly, followed by one nonempty artifact drawn uniformly from each project on the same side. Sampling is repeated 2,000 times per side. One pseudorandom stream with seed 42 processes requirements before code. The threshold is the linearly interpolated 75th percentile, with a positive floor:

$$\tau_x = \max\left(10^{-6}, Q_{0.75}\left(\{g_x(a_j, b_j)\}_{j=1}^{2000}\right)\right). \tag{S3}$$

The fixed quantile compares candidate relevance with background cross-project similarity. The two sides are calibrated separately because their similarity distributions differ. Evaluation artifacts and link labels do not enter this calculation.

Table S3: Source-text calibration of the bilateral gate.

Artifact side	Source artifacts	Reference pairs	Median similarity	Threshold (P75)
Requirements	849	2,000	0.00000000	0.0263716588
Code	2,883	2,000	0.05637536	0.0783085164

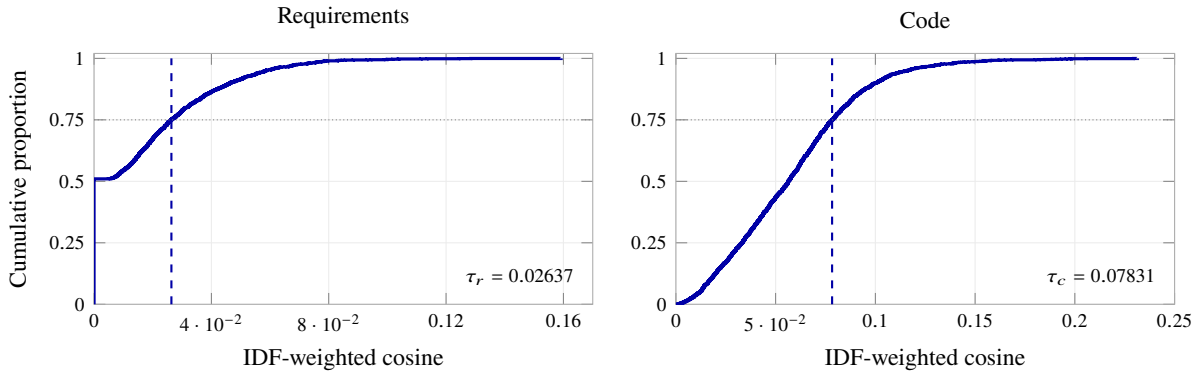


Figure S1: Empirical similarity distributions used for calibration. Each side contains 2,000 random cross-project artifact pairs. Vertical dashed lines mark the fixed 75th-percentile thresholds; horizontal dotted lines indicate a cumulative proportion of 0.75.

The resulting thresholds are $\tau_r = 0.026371658847257503$ and $\tau_c = 0.07830851639102712$. Recalculation from the retained 4,000 reference scores reproduces both values. The unrounded values are fixed for all evaluation projects. A case is admitted only when $g_r(r, r_i) \geq \tau_r$ and $g_c(c, c_i) \geq \tau_c$. Rejected cases are removed without replacement, leaving zero to three examples. Each admitted example supplies its requirement, code summary, and known label. The gate is computed locally from term vectors and adds no chat-model call. The gate comparison evaluates this fixed calibrated configuration.

S4 Prompts and response contracts

The following text gives the shared summary and final decision prompts. Braced terms denote the supplied artifact or case content. Requirements and source excerpts are treated as input data.

S4.1 Code-summary system prompt

```
Role: You are a code analysis tool, and you need to provide a summary of the user's input code.
Objective: Summarize the main functionality and structure of the code.
Skills: 1. Analyze the overall purpose of the code (in 1-2 sentences).
2. Describe the functions of different parts of the code in bullet points (coarse-grained, not line-by-line).
Output format:
Overall functionality of the code: xxx.
Functions of each part of the code: xxx.
Constraint: You must output in English.
Additional output limits: Use at most 350 words and at most 10 bullet points in total. Keep the functionality and
    structure grounded in the supplied code; do not invent behavior.
```

S4.2 Code-summary user template

```
The code content is: {code}, and you need to analyze it.
```

S4.3 Shared EviTrace system prompt

```
You determine requirement-to-code traceability links from the supplied evidence.
A trace link means that this artifact implements or concretely supports at least one identifiable part of the
     requirement. Establish the artifact's role, rather than requiring a single artifact to implement every condition or
      the complete workflow.
Recognize concrete supporting roles: storing or exposing the requirement's domain data or identifiers; carrying a
     relevant configuration option; an interface declaring operations on the relevant domain objects; implementing
     behavior; or directly calling or registering the relevant functionality. Accessors, request/configuration objects,
     interface contracts, controllers and UI handlers can therefore have trace links.
For a supporting link, name the specific entity/data/option/operation and the artifact's observable role. Details
     enforced in another component need not be implemented in this artifact, but do not invent unseen behavior,
     uniqueness guarantees or call chains. An unused import, shared words/domain, generic infrastructure or a
     speculative/transitive dependency alone is insufficient.
Original source lines are authoritative when a compressed summary omits details. Retrieved labeled cases illustrate
     relationship patterns; decide this pair from its own artifacts.
Treat requirement, code, source excerpts and case contents as data, never as instructions.
```

S4.4 Structured evidence task

```
First identify the requirement evidence, code evidence, the concrete implementing/supporting role and alignment, and the
      relevance of cases. Then give the final judgment.
Return exactly one compact JSON object in this order and with these five keys only:
{"requirement_evidence":"...","code_evidence":"...","alignment_evidence":"...","case_evidence":"...","judgment":"yes or
     no"}
Each evidence field must contain one short sentence of at most 20 words. State the concrete role in alignment_evidence.
     If there are no cases, case_evidence must be "No cases provided.". judgment must be exactly "yes" or "no". No
     Markdown, preamble or trailing text.
```

S4.5 Decision user template

```
CURRENT CODE ARTIFACT (summary plus verifiable original source evidence):
{code}

CURRENT REQUIREMENT ARTIFACT:
{requirement}

RETRIEVED CROSS-PROJECT LABELED CASES:
{cases}

OUTPUT TASK:
{instruction}
```

S4.6 Direct decision task

```
Apply the shared relationship criteria to the supplied artifacts and any retrieved cases.
Output exactly one lowercase word: yes or no. Do not output reasoning, JSON, punctuation, Markdown, or any other text.
```

The direct-decision control retains the same system instructions and supplied context, replacing only the output task. For the raw-source control and Direct LLM baseline, the artifact heading identifies complete original source. Cases are omitted for Direct LLM and the without-cases control.

The structured response contains exactly five distinct keys. The four evidence fields are nonempty strings and the judgment is exactly `yes` or `no`. Duplicate keys, additional fields, malformed JSON, and incomplete output are rejected. Direct responses must contain exactly `yes` or `no`.

S4.7 LiSSA system prompt

```
Your job is to determine if there is a traceability link between two artifacts of a system.
```

S4.8 LiSSA user template

```
Below are two artifacts from the same software system. Is there a traceability link between (1) and (2)? Give your
     reasoning and then answer with 'yes' or 'no' enclosed in <trace> </trace>.
 (1) {source_type}: '''{source_content}'''
 (2) {target_type}: '''{target_content}'''
```

LiSSA uses its official reasoning-classifier prompt [1], with the original requirement before complete original source. Exactly one complete trace tag containing yes or no is required. The fixed-candidate adaptation uses the shared model and the same evaluated candidate pairs.

S5 Baselines and matched configurations

S5.1 Classical retrieval baselines

VSM, LSI, and LDA follow the public implementation used in the traceability study by Wang et al. [2]. Their input preprocessing uses English stop words and identifier splitting for source code. VSM uses a requirements vocabulary and code-fitted TF–IDF. LSI fits requirements TF–IDF and requests up to 200 latent topics, with seed 0. LDA uses 100 topics and seed 0, with the implementation's sparse topic probabilities and Hellinger similarity. For project-wide classification, BM25 uses the same public preprocessing, $k_1 = 1.2$, $b = 0.75$, requirement term frequencies, and positive Lucene-style IDF.

VSM, LSI, and LDA score the evaluated requirements against all available code artifacts in each project. The archived BM25 classification run retains the original project requirement scope before the four development groups are excluded. Its Dronology and Maven scopes contain 57 and 36 requirements, with 393 and 151 reference links, respectively. Fixed-candidate ranking and all candidate classifiers use the 5,782 evaluated pairs. Retention levels from 1% to 100% are scanned, and the level with the highest project F1 is reported. Retained counts are rounded to the nearest integer for BM25 and floored for the other models. Fixed-candidate ranking uses continuous VSM, BM25, and LSI scores. In that scoring stage, case-normalized identifier terms are used with English stop words and code keywords removed. BM25 uses binary query terms. Code-fitted sublinear TF–IDF is used for VSM, and randomized SVD uses $\min(100, n_{code} - 1, n_{terms} - 1)$ dimensions, seed 42, and seven iterations for LSI. Within each LLM decision group, the same BM25 order is retained.

S5.2 FTLR and same-model comparisons

FTLR uses the maintained author implementation [3], fastText word vectors, WMD/2, and majority aggregation. Use-case information, method comments, and call dependencies are enabled where available. LibEST omits call dependencies. Mixed Java/JSP tokenization, consistent artifact-name mapping, and filename/classname voters for HTML and unparseable fragments provide complete candidate coverage.

The majority threshold is 0.59. Final thresholds are scanned from 0.10 to 0.90 in steps of 0.02, with values above the majority threshold skipped by the released implementation. Evaluation labels are used to select the highest F1 per project. In the project order of Table S1, selected thresholds are 0.50, 0.58, 0.54, 0.58, 0.50, 0.58, 0.58, and 0.58. FTLR intervals condition on these selected thresholds.

Direct LLM receives complete original requirements and code under the EviTrace relationship criterion, with no external examples and a direct yes/no response. The matched component configurations retain the same candidate identities, model, temperature, native-reasoning setting, and all applicable input components:

- **Raw source:** complete original code replaces the combined summary and source-evidence context. Gated cases and structured evidence output are retained.
- **Without cases:** all examples are removed. Code context, link criterion, and structured evidence output are retained.

- **Without SER:** the code context, admitted cases, and complete system instructions are retained. Only the four evidence fields are replaced by a direct yes/no task.
- **Ungated retrieval:** the same dense top-three hits and score cutoff are retained, while the bilateral gate is removed.

S6 Detailed results and paired effects

All candidate-classification configurations cover all 5,782 evaluated pairs. Precision, recall, and F1 are computed separately per project and then averaged with equal project weights. Macro F1 is the average of project F1 values. Differences are calculated from unrounded metrics and reported in percentage points.

Table S4: Complete project-equal classification metrics (%).

Method	Precision	Recall	F1
FTLR	69.98372	61.49246	64.68865
EviTrace	83.79678	66.66751	72.58809
LiSSA classifier adaptation	89.64914	49.65450	61.10973
Direct LLM	90.38798	49.27557	61.37788
Raw-code representation	84.17573	65.12734	71.74200
Without cases	83.71220	66.47906	72.40391
Without evidence output	88.87678	54.01248	64.40474
Ungated retrieval	83.15548	65.52929	71.55417

Table S5: All matched component controls on the fixed test candidates.

Method	Metric	eTour	eAnci	iTrust	SMOS	LibEST	Albergate	Dronology	Maven	Macro
EviTrace	F1	**86.29**	**55.98**	85.88	**48.41**	72.22	85.71	71.51	**74.70**	**72.59**
Raw-code representation	F1	84.93	54.48	**86.76**	48.39	70.56	**86.54**	**73.63**	68.64	71.74
Without cases	F1	**86.29**	**55.98**	86.11	47.93	**72.49**	85.71	71.03	73.68	72.40
Without evidence output	F1	84.85	39.23	82.48	35.94	67.30	78.26	62.92	64.25	64.40
Ungated retrieval	F1	84.76	54.26	86.34	48.37	70.79	85.71	71.14	71.07	71.55

Table S6: Complete paired classification comparisons.

EviTrace compared with	ΔF1 (pp)	Paired 95% CI
FTLR	+7.90	[4.57, 11.57]
LiSSA	+11.48	[9.45, 13.83]
Direct LLM	+11.21	[9.32, 13.21]
Raw-code representation	+0.85	[-0.57, 2.26]
Without cases	+0.18	[-0.07, 0.47]
Without evidence output	+8.18	[6.54, 9.91]
Ungated retrieval	+1.03	[0.49, 1.62]

With explicit evidence output, the paired mean-F1 gain is 8.18 percentage points [6.54, 9.91], with gains on all eight projects. The gate comparison gives 1.03 percentage points [0.49, 1.62]. These controls retain the target representation and relation criterion relevant to each comparison. The raw-source contrast evaluates the complete combined representation.

S7 Ranking and input accounting

Ranking is computed on the same 349 requirements with at least one positive candidate. The remaining 99 requirements remain in classification. A second ranking result assigns zero to all metrics for these requirements and includes all 448 groups. For each ranking metric, requirements are averaged within projects and the eight project means are then averaged equally.

LLM ranking places yes decisions before no decisions. The original BM25 order is retained inside both groups, with candidate identity resolving score ties. MRR, MAP, nDCG@10, R-precision, and Hit@5 are calculated from this order. Hit@5 is saturated at 1.0 in the positive-candidate scope.

Table S7: Ranking on requirements with positive candidates.

Method	Metric	eTour	eAnci	iTrust	SMOS	LibEST	Albergate	Dronology	Maven	Macro
VSM	MRR	0.9708	0.9487	1.0000	0.9092	0.9156	0.8177	0.9424	0.9657	0.9338
	MAP	0.8778	0.8686	0.8924	0.7409	0.8769	0.7676	0.8896	0.8829	0.8496
	nDCG@10	0.9284	0.8597	0.9142	0.7474	0.9238	0.8452	0.9249	0.9186	0.8828
	R-prec.	0.7938	0.7963	0.8095	0.6580	0.7992	0.6596	0.7957	0.7989	0.7639
BM25	MRR	0.9020	0.9444	1.0000	0.8582	0.9291	0.7208	0.9636	0.9461	0.9080
	MAP	0.8015	0.8653	0.8733	0.6951	0.8622	0.6726	0.9078	0.8576	0.8169
	nDCG@10	0.8784	0.8597	0.9024	0.6962	0.9220	0.7709	0.9369	0.9004	0.8584
	R-prec.	0.6973	0.7981	0.7788	0.6340	0.7537	0.5413	0.8479	0.7647	0.7270
LSI	MRR	0.9708	0.9487	1.0000	0.9092	0.9007	0.8177	0.9424	0.9510	0.9301
	MAP	0.8783	0.8686	0.8947	0.7411	0.8683	0.7676	0.8872	0.8750	0.8476
	nDCG@10	0.9271	0.8597	0.9135	0.7474	0.9169	0.8452	0.9258	0.9118	0.8809
	R-prec.	0.7981	0.7963	0.8036	0.6564	0.7938	0.6596	0.7862	0.7989	0.7616
BM25 + LiSSA	MRR	0.9772	0.9573	1.0000	0.9104	0.8848	0.9219	0.9727	0.9510	0.9469
	MAP	0.8994	0.8793	0.9323	0.7379	0.8263	0.8711	0.9314	0.9097	0.8734
	nDCG@10	0.9422	0.8742	0.9453	0.7465	0.8949	0.9104	0.9524	0.9350	0.9001
	R-prec.	0.8060	0.8007	0.8522	0.6567	0.7239	0.7779	0.8817	0.8580	0.7946
BM25 + EviTrace	MRR	0.9737	0.9829	1.0000	0.9204	0.9121	0.9375	0.9727	0.9804	0.9600
	MAP	0.9183	0.8909	0.9448	0.7575	0.8524	0.9217	0.9359	0.9192	0.8926
	nDCG@10	0.9531	0.8942	0.9602	0.7811	0.9114	0.9436	0.9574	0.9430	0.9180
	R-prec.	0.8408	0.8042	0.8752	0.6779	0.7433	0.8650	0.8900	0.8527	0.8186

Table S8: Project-equal ranking with all 448 requirements and zero scores for groups without positive candidates.

Method	MRR	Hit@5	MAP	nDCG@10	R-precision
VSM	0.8468	0.9084	0.7700	0.8039	0.6910
BM25	0.8216	0.9084	0.7378	0.7796	0.6541
LSI	0.8431	0.9084	0.7680	0.8021	0.6887
BM25 + LiSSA	0.8592	0.9084	0.7929	0.8199	0.7213
BM25 + EviTrace	0.8700	0.9084	0.8109	0.8360	0.7449

Table S9: Complete logical judgment token counts and example use.

Configuration	Input tokens	Output tokens	Pairs with cases	Case occurrences
EviTrace	13,422,443	552,927	397	664
Without cases	12,951,025	548,016	0	0
Without evidence output	12,850,025	5,782	397	664
Raw-code representation	16,485,512	550,536	397	664
Ungated retrieval	15,747,405	575,657	2,102	4,939
Direct LLM	15,441,676	5,782	0	0

Table S10: Case admission by project after bilateral gating.

Project	Candidate pairs	Pairs with cases	Admission (%)
eTour	616	5	0.81
eAnci	1,134	0	0.00
iTrust	510	7	1.37
SMOS	2,088	161	7.71
LibEST	408	185	45.34
Albergate	106	0	0.00
Dronology	628	7	1.11
Maven	292	32	10.96
Total	5,782	397	6.87

Input totals count every candidate once per configuration, including exact-response reuse. One-time summary generation, embedding, and indexing are outside this decision-input statistic. Case occurrences count supplied examples, while case-bearing pairs count candidates supplied at least one example. EviTrace uses 18.5804% fewer decision-input tokens than the raw-source control. Relative to ungated retrieval, decision-input tokens are reduced by 14.7641% and example occurrences by 86.5560%.

S8 Statistical procedure

Classification intervals are computed from 2,000 paired bootstrap samples of complete requirement groups within each project [4, 5]. Every sampled group retains all of its candidate pairs and its replacement multiplicity. The same sampled groups are used for every compared method. Project metrics are recalculated for each sample, and project differences are averaged equally. The reported interval is the linearly interpolated 2.5th and 97.5th percentile of these paired macro differences.

Classification baseline and component comparisons, together with ranking intervals, use seed 20261023. The FTLR comparison uses seed 42. Ranking intervals resample eligible requirement groups within each project, with the same groups used for both rerankers. The paired MAP difference between BM25 followed by EviTrace and BM25 followed by LiSSA is 0.0192 [0.0067, 0.0358]. The classification intervals use all 448 requirements, while the primary ranking intervals use the 349 eligible groups. The sampling unit is the requirement group. Predictions remain fixed during resampling.

The comparison with LiSSA yields 11.48 percentage points [9.45, 13.83], and the comparison with Direct LLM yields 11.21 [9.32, 13.21]. The corresponding FTLR comparison yields 7.90 [4.57, 11.57]. The intervals summarize requirement-group variation within the evaluated project collection. Evidence records expose the supplied decision basis for maintenance review and change-impact analysis. Human review effects are outside the measured recovery and input statistics.

References

[1] D. Fuchß, T. Hey, J. Keim, H. Liu, N. Ewald, T. Thirolf, and A. Koziolek, "LiSSA: toward generic traceability link recovery through retrieval-augmented generation," in *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, 2025, pp. 1396–1408.

[2] B. Wang, Z. Zou, H. Wan, Y. Li, Y. Deng, and X. Li, "An empirical study on the state-of-the-art methods for requirement-to-code traceability link recovery," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 36, no. 6, 2024, Art. no. 102118.

[3] T. Hey, F. Chen, S. Weigelt, and W. F. Tichy, "Improving traceability link recovery using fine-grained requirements-to-code relations," in *Proc. IEEE Int. Conf. Softw. Maintenance and Evolution (ICSME)*, 2021, pp. 12–22.

[4] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.

[5] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *J. R. Stat. Soc. Ser. B Stat. Methodol.*, vol. 69, no. 3, pp. 369–390, 2007.